МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод хорд.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 5. Функция $f(x) = arctg(1/x) - x^2$.

Изучить метод бисекции для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Теоретический минимум.

Пусть найден отрезок $[a, b]$, на котором функция $f(x)$ меняет знак. Для определенности положим $f\ (a) > 0, f\ (b) < 0$. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения $f(x) = 0$ принимаются значения $c0, c1, \ldots$ точек пересечения хорды с осью абсцисс, как это показано на рис.1.

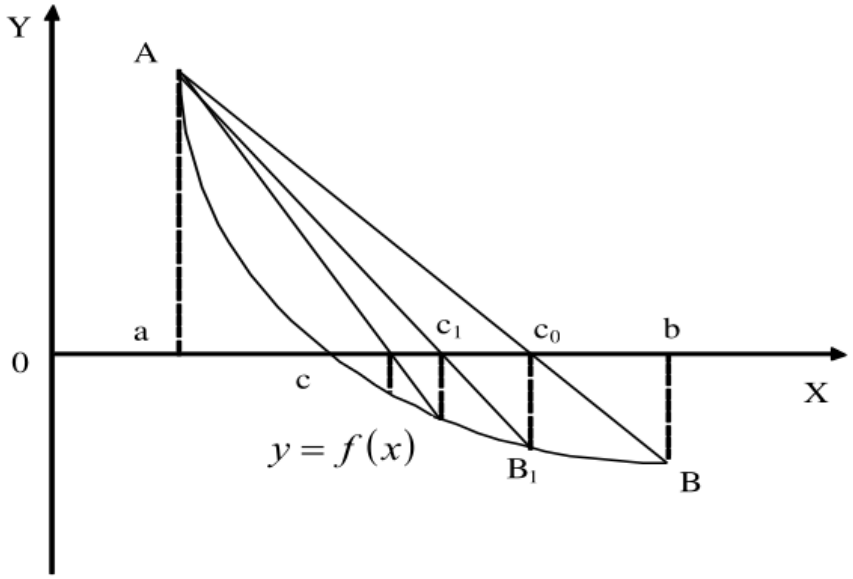

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y\ -\ f(a)}{f(b) - f(a)} = \frac{x - a}{b - a}$$

Для точки пересечения ее с осью абсцисс (x=c0, y=0) получается уравнение $c0 = a - \frac{b-a}{f(b)-f(a)} f(a)$.

Далее сравниваются знаки величин $f\ (a)$ и $f\ (c0)$ и для рассматриваемого случая оказывается, что корень находится в интервале (a, c0), так как

$f\ (a)f(c0) < 0$. Отрезок [c0,b] отбрасывается. Следующая итерации состоит в определении нового приближения c1 как точки пересечения хорды AB1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение $f\ (cn)$ не станет по модулю меньше заданного числа

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

Выполнение работы.

1) Графический способ. С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка $[a, b]$ близкого к $f(x) = 0$ на котором выполняется условие $f(a) * f(b) < 0$.

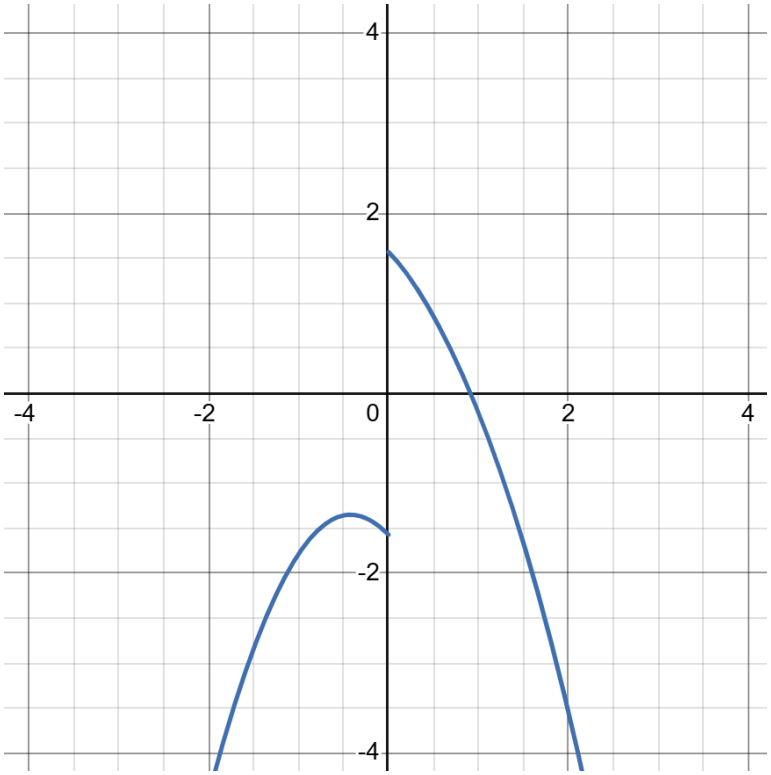

Рисунок 1 - график функции $f(x) = arctg(1/x) - x^2$

На графике видно, что корень находится около значения x≈0.915. Можно взять отрезок, где a∈(0.1,0.91), а b∈(0.92,2.0). Для простоты и надежности возьмем отрезок [0.5,1.5]. Проверка: f(0.5)≈0.857>0, f(1.5)≈−1.662<0. Условие выполняется.

Аналитический способ (аналитическое отделение корня):

Для доказательства наличия и единственности корня на выбранном отрезке исследуем функцию аналитически с помощью производной. Найдем первую производную функции: $f`(x) = -1/(x^2+1) – 2x$.

Рассмотрим поведение производной на интервале $(0,+\infty)$. Очевидно, что при любых $x>0$ выражения $(-1/(x^2+1))$ и $(-2x)$ строго меньше нуля. Следовательно, $f'(x) < 0$ при $x\in(0,+\infty)$. Так как производная знакопостоянна и отрицательна, функция $f(x)$ монотонно и строго убывает на всем интервале. Это гарантирует, что функция может пересечь ось Ox не более одного раза (корень единственный).

Проверим выполнение условий теоремы Коши (Больцано-Коши) на границах выбранного отрезка [a,b]=[0.5,1.5]:

- $f(0.5)=\mathrm{arctg}(1/0.5)-0.5^2=\mathrm{arctg}(2)-0.25\approx1.107-0.25=0.857$
- $f(1.5)=\mathrm{arctg}(1/1.5)-1.5^2=\mathrm{arctg}(2/3)-2.25\approx0.588-2.25=-1.662$

Поскольку функция $f(x)$ непрерывна на отрезке [0.5,1.5] и принимает на его концах значения разных знаков ($f(0.5)>0$, а $f(1.5)<0$, следовательно $f(a)\cdot f(b)<0$), то на этом отрезке существует единственный корень. Аналитическое отделение корня успешно выполнено.

2) Функция `extern double F(double x)` теперь возвращает значение `std::atan(1.0, x) - (x * x)` в соответствии с вариантом $f(x) = arctg(1/x) - x^2$. Исходный код в Приложении А.
3) Создана функция, находящаяся в файле main.cpp в Приложении А.
4) Проведены вычисления, построен график (Рисунок 2).

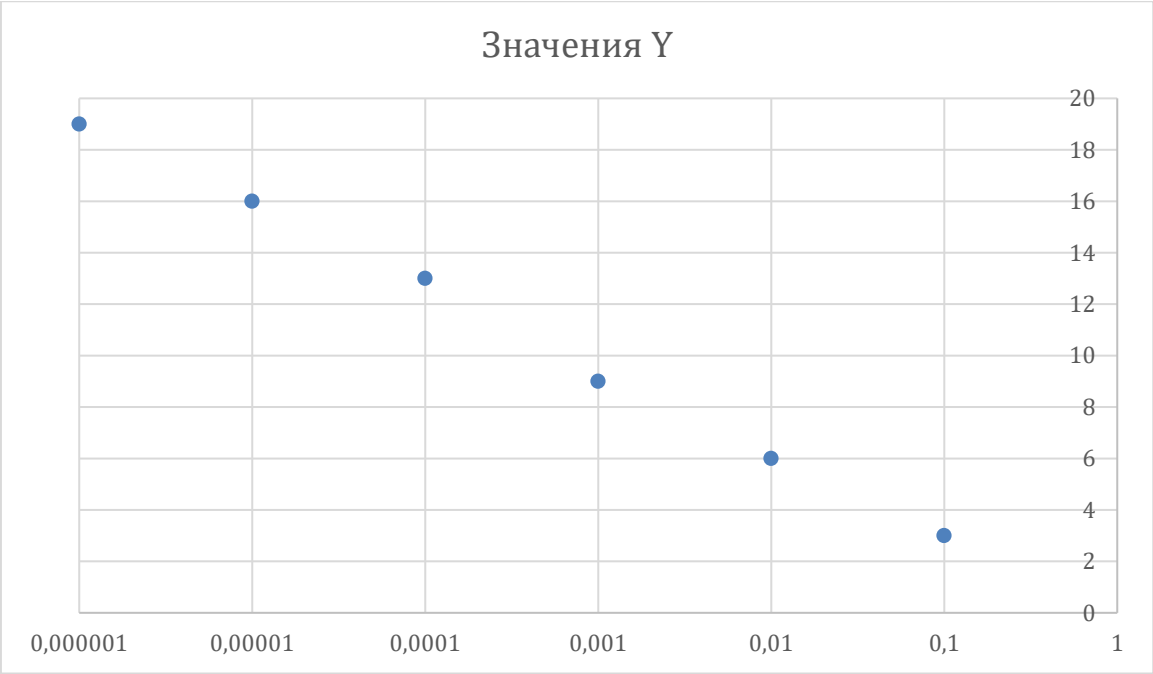

Рисунок 2 - “График” зависимости количества итераций от точности вычислений

Можно заметить, что на Рисунке 2 с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности добавлены флаги компиляции ROUND и DELTA. При ROUND=1 DELTA задается автоматически значением 0.001, если нужно можно задать отдельно. Ниже в Таблице 1 приведены значения при разных дельта.

Таблица 1 - исследование чувствительности метода к ошибкам в исходных данных

Значение DELTA	ε	Значение возвращаемое функцией HORDA	Количество итераций необходимых для заданной ε.
10^{-1}	10^{-1}	0.91363	2
	10^{-2}	0.91363	2
	10^{-3}	0.91363	2

	10^{-4}	0.91363	2
	10^{-5}	0.91363	2
	10^{-6}	0.91363	2

Продолжение Таблицы 1

Значение DELTA	ε	Значение возвращаемое функцией HORDA	Количество итераций необходимых для заданной ε.
10^{-3}	10^{-1}	0.84021	1
	10^{-2}	0.89985	2
	10^{-3}	0.91412	4
	10^{-4}	0.91501	5
	10^{-5}	0.91501	5
	10^{-6}	0.91501	5
10^{-5}	10^{-1}	0.84024	1
	10^{-2}	0.89993	2
	10^{-3}	0.91005	3
	10^{-4}	0.91461	5
	10^{-5}	0.91515	7
	10^{-6}	0.91520	8
10^{-7}	10^{-1}	0.84024	1
	10^{-2}	0.89993	2
	10^{-3}	0.91006	3
	10^{-4}	0.91462	5
	10^{-5}	0.91515	7
	10^{-6}	0.91521	10

Выводы.

В ходе работы был изучен метод хорд. Для уравнения $f(x) = arctg(1/x) - x^2$ корень был успешно найден на отрезке [0.5,1.5] с точностью от 0.1 до 0.000001. Построенный график показал логарифмическую зависимость числа итераций от точности: увеличение точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод бисекции является устойчивым к малым ошибкам округления значений функции, однако при грубых возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <cmath>
#include "methods.hpp"

#ifndef LEFT
    #define LEFT 0.5 // левая граница
#endif
#ifndef RIGHT
    #define RIGHT 1.5 // правая граница
#endif

int main() {
    int iterationsCount;
    std::cout << "Eps\t\t | Root\t\t\t | Iterations\n";
    std::cout << "--------------------------------------------------\n";
    for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
        std::cout << epsilon << "\t\t | "
                  << HORDA(LEFT, RIGHT, epsilon, iterationsCount) << "\t\t | "
                  << iterationsCount << '\n';
    }
    return 0;
}
```

Файл methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#ifndef ROUND_FLAG
    #define ROUND_FLAG 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND_FLAG
extern double F(double x) {
    // Вариант 5: f(x) = arctg(1/x) - x^2
    return atan(1.0 / x) - (x * x);
}
#else
extern double F(double x) {
    return Round(atan(1.0 / x) - (x * x), DELTA);
}
#endif

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;

    if (FLeft * FRight > 0.0) {
        puts("Неверное задание интервала\n");
        exit(1);
    }
    if (Eps <= 0.0) {
```

```
        puts("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0) return (X);

        if (Y * FLeft < 0.0)
        {
            Right = X;
            FRight = Y;
        }
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    while ( fabs(Y) >= Eps );

    return(X);
}

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;
    if (FLeft * FRight > 0.0) {puts("Неверное задание интервала\n");exit(1);}
    if (Eps <= 0.0) {puts("Неверное задание точности\n");exit(1);}
    N=0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;
    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left);
        Y = F(X);
        if (Y == 0.0) return (X);
        if (Y * FLeft < 0.0) Right = X;
        else { Left = X; FLeft = Y; }
        N++;
    };
    return(X);
}

double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else         return (Delta * (long((X / Delta) - 0.5)));
```

```
}

double ITER(double X0, double Eps, int &N)
{
    if (Eps <= 0.0) {puts("Неверное задание точности\n"); exit (1);}
    double X1 = F(X0);
    double X2 = F(X1);
    N = 2;
    while( (X1 - X2)*(X1 - X2) > fabs((2*X1 - X0 - X2)*Eps) )
    {
        X0 = X1;
        X1 = X2;
        X2 = F(X1);
        N++;
    }
    return(X2);
}

#ifdef __NEWTON
double NEWTON (double X, double Eps, int &N)
{
    extern double F1 (double);
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0) return (X);
        Y1 = F1(X);
        if (Y1 == 0.0) {puts("Производная обратилась в ноль\n"); exit(1);}
        DX = Y / Y1; X = X - DX; N++;
    }
    while (fabs(DX) > Eps);
    return (X);
}
#endif
```

Файл methods.hpp

```
#ifndef METHODS_HPP
#define METHODS_HPP

extern double F(double);

#ifdef __NEWTON
extern double F1(double);
#endif

double Round(double X, double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
double ITER(double X0, double Eps, int &N);
double HORDA(double Left, double Right, double Eps, int &N);
double NEWTON(double X, double Eps, int &N);

#endif // METHODS_HPP
```